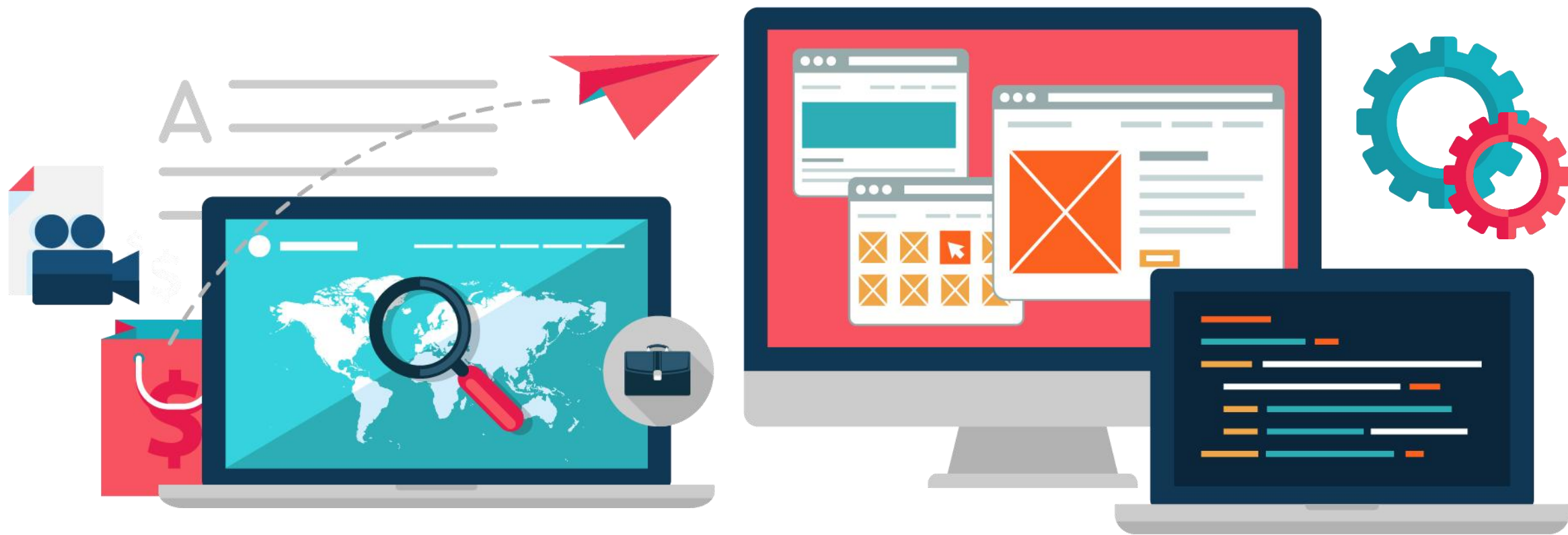


面向对象程序设计

Object Oriented Programming





虚机制



静态编联与动态编联

虚函数

虚函数的作用机制

虚函数的作用机制（例）

虚函数的访问

具体类和抽象类

运行时类型识别(RTTI)





虚机制的引入

```
class shape{  
public:  
    void show( ) const {  
        cout<<“面积是:”<<Area( )<<endl;  
    }  
    float Area( ) const { return 0; }  
};
```

```
class Drawer{  
public:  
    void paint(Shape & shape){  
        shape.show();  
    }  
};
```

```
int main( ) {  
    Rectangel rect(1,2);  
    Circle      cir(1);  
    Drawer      drawer;  
    drawer.paint(rect);    //输出是0,为什么?  
    drawer.paint(cir);     //输出是0,为什么?  
}
```

```
class Rectangle: public Shape {  
public:  
    Rectangle(float w,float h) {  
        mWidth = w; mHeight = h;  
    }  
    float Area( ) const {  
        return mWidth*mHeight;  
    }  
private:  
    float mWidth, mHeight;  
};
```

```
class Circle: public Shape {  
public:  
    Circle(float r): mRadius( r ) { }  
    float Area( ) const {  
        return mRadius * mRadius*3.14;  
    }  
private:  
    float mRadius;  
};
```



```
class Tamer //驯养员类
{
public:
    Tamer( ) { /*略*/ }
    ~Tamer( ) {
        /*略*/
    }
    int feed( Pet & pet)
        { pet.eat( ); }
private: /*略*/
};
```

```
int main( ) {
    Tamer tamer;
    Dog dog;
    Snake snake;
    tamer.feed(dog); //输出“Pet Normal Eat! ”
    tamer.feed(snake); //输出“Pet Normal Eat! ”
}
```

```
class Pet {
public:
    ~Pet ( ) { }
    void eat( ){
        cout << “Pet Normal Eat! ” ;
    }
    //....
};
class Dog: public Pet{
    void eat(){
        cout << “Dog Bite Eat! ” ;
    }
};
class Snake: public Pet{
    void eat(){
        cout << “Snake Swallow Eat! ” ;
    }
};
```



◆ 静态编联(早绑定, 静态绑定):

- ◆ 编译期间就决定了程序运行时将具体调用哪个 **函数体**。即使没有主程序, 也能知道程序中各个 **函数体** 之间的调用关系。

- ◆ 例上页: `Pet& pet = dog;`
`pet.eat( );`

◆ 动态编联(晚绑定, 动态绑定):

- ◆ 在运行期间, 决定具体调用哪个 **函数体**。

◆ 动态编联的实现

- ◆ 多种方式

- ◆ 虚机制 (使用虚拟函数和虚拟函数表)



```
class Tamer //驯养员类
{
public:
    Tamer( ) { /*略*/ }
    ~ Tamer( ) {
        /*略*/
    }
    int feed( Pet & pet)
        { pet.eat( ); }
private: /*略*/
};
```

```
int main( ) {
    Tamer tamer;
    Dog dog;
    Snake snake;
    tamer.feed(dog); //输出“Dog Bite Eat! ”
    tamer.feed(snake); //输出“Snake Swallow Eat! ”
}
```

```
class Pet {
public:
    virtual ~Pet ( ) {}
    virtual void eat( ){
        cout << “Pet Normal Eat! ” ;
    }
    //....
};
class Dog: public Pet{
    virtual void eat(){
        cout << “Dog Bite Eat! ” ;
    }
};
class Snake: public Pet{
    virtual void eat(){
        cout << “Snake Swallow Eat! ” ;
    }
};
```



虚机制的引入

```
class shape{
public: virtual ~Shape( ) { }
    void show( ) const {
        cout<<"面积是:"<<Area( )<<endl;
    }
    virtual float Area( ) const { return 0; }
};
```

```
class Drawer{
public:
    void paint(Shape & shape){
        shape.show();
    }
};
```

```
int main( ) {
    Rectangel rect(1,2);
    Circle cir(1);
    Drawer drawer;
    drawer.paint(rect); //输出是2
    drawer.paint(cir); //输出是3.14
}
```

```
class Rectangle: public Shape {
public:
    Rectangle(float w,float h) {
        mWidth = w; mHeight = h;
    }
    virtual float Area( ) const {
        return mWidth*mHeight;
    }
private:
    float mWidth, mHeight;
};
```

```
class Circle: public Shape {
public:
    Circle(float r): mRadius( r ) { }
    virtual float Area( ) const {
        return mRadius * mRadius*3.14;
    }
private:
    float mRadius;
};
```



- ◆ 虚函数的格式(必须是非静态成员函数)
 - ◆ 声明: **virtual** 返回类型 函数名(参数列表)[const];
 - ◆ 定义: 同一般成员函数, 可内联或外联实现
- ◆ 虚函数说明:
 - ◆ 必须是成员函数
 - ◆ 静态成员函数和构造函数、拷贝构造函数不能是虚的
 - ◆ 析构函数可以是虚的, 若有派生类, 则应该是虚的
 - ◆ 若类中有其它虚函数, 那么析构函数也应该是虚的
 - ◆ 赋值函数通常不定义为虚的
 - ◆ 虚函数可以带const修饰, 也可以不带
 - ◆ 访问控制可以任意(public、protected、private)
 - ◆ 类外定义时不加virtual关键字(同static), 否则编译错误

- ◆ 通常采用public继承方式
- ◆ 继承自基类的虚函数（除虚的析构函数）
 - ◆ 若基类的析构函数是虚的，那么派生类中的析构函数也是虚的
- ◆ 派生类中新定义的虚函数
- ◆ 派生类override基类中虚函数(或复写、重写、覆盖)
 - 函数名字同基类中虚函数的名字
 - virtual关键字可省略
 - 返回类型必须与基类中虚函数的返回类型相同或相容
 - 可能会隐藏基类中重载的虚函数(overwrite,hide)
 - 可选override说明符

```
class Parent {  
public:  
    virtual ~Parent ( ) = default;  
    virtual void F( );  
    int Normal( );  
protected:  
    virtual Parent * G( );  
};
```

```
class Child: public Parent {  
public:  
    virtual ~Child ( ) = default;  
    virtual void F( );  
protected:  
    virtual Child * G( );  
};
```

```
struct A {    virtual void foo();  
              void bar();  
};  
struct B : A{  
    void foo() const override; // 错误  
    void foo() override; // OK  
    void bar() override; // 错误  
};
```



```
class Parent {  
public:  
    Parent() ;  
    virtual ~Parent ( ) { }  
    void F( );  
    virtual void G( );  
protected:  
    virtual void K( );  
};
```

Parent类的3个虚函数：

1. Parent:: ~Parent()
2. Parent:: G()
3. Parent:: K()

```
class Child:public Parent {  
public:  
    Child() ;  
    virtual ~Child ( ) { }  
    void Other();  
    virtual void G( );  
protected:  
    virtual void H( );  
};
```

Child类的4个虚函数：

1. Child::~~Child()
2. Child::G() //Child中Override基类的G()
3. Parent::K() //继承自父类的K()
4. Child::H() //子类中新定义的H()

Parent类的3个虚函数：

1. Parent:: ~Parent()
2. Parent:: G()
3. Parent:: K()

Parent类的虚函数表

```
&Parent::~Parent()  
&Parent::G()  
&Parent::K()
```

Child类的4个虚函数：

1. Child::~~Child()
2. Child::G() //Child中Override基类的G()
3. Parent::K() //继承自父类的K()
4. Child::H() //子类中新定义的H()

Child类的虚函数表

```
&Child::~~ Child()  
&Child ::G()  
&Parent::K()  
&Child::H()
```

- ◆ 虚函数表(虚拟表、虚表、VTable):
 - ◆ 一个指针数组，各元素存放对应虚函数的入口地址
- ◆ 说明：
 - ◆ 要求对应的类中至少有一个虚函数
 - ◆ 一个类至多有一个虚拟表，同一个类的不同对象共享该虚拟表
 - ◆ 首次创建该类实例对象时，在内存中同时创建该类的虚拟表
 - ◆ 按照函数顺序的序号依次存放入口地址

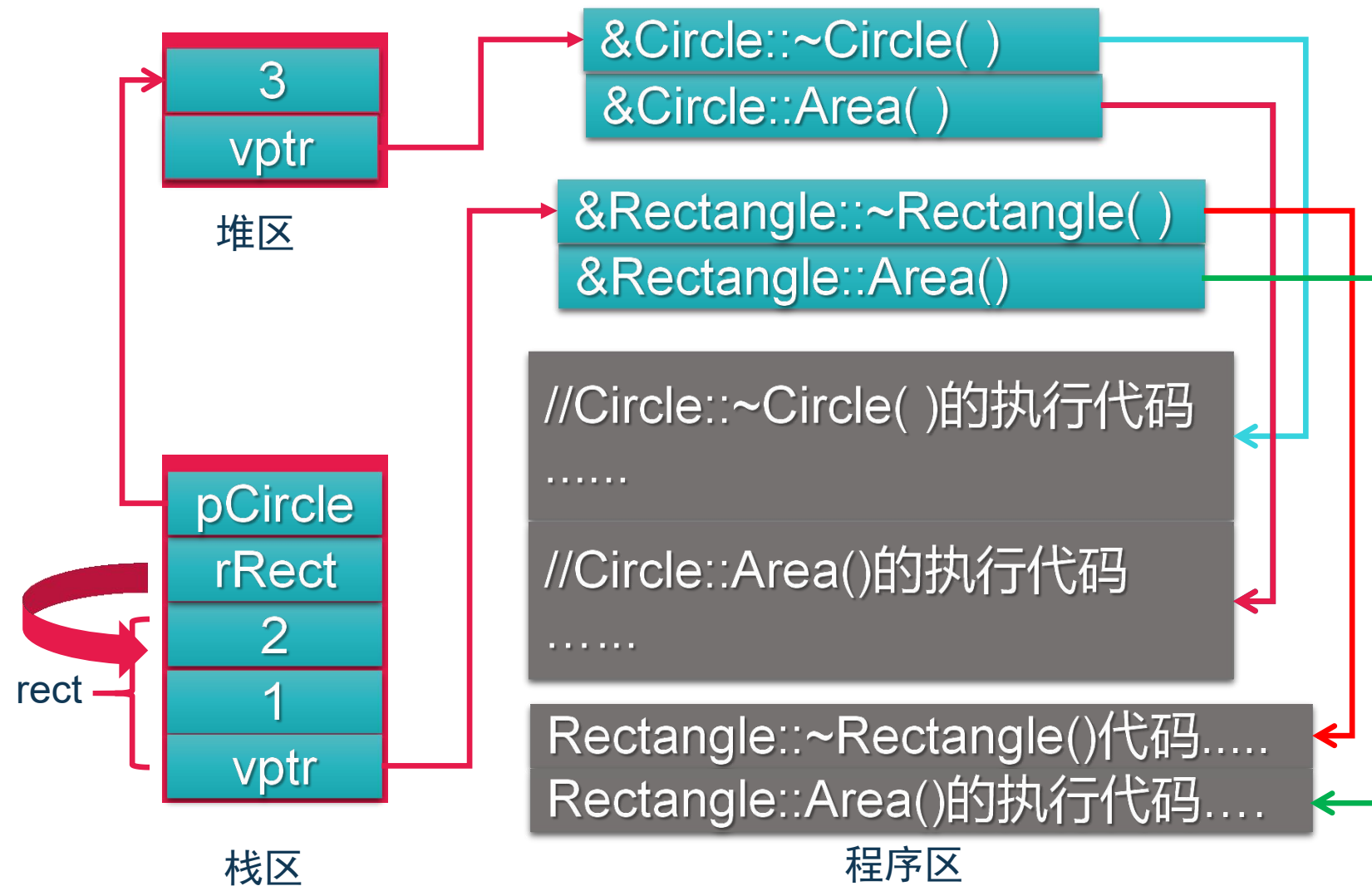


```
class Shape{
public:
    virtual ~Shape( ) { }
    void Show( ) const {
        cout<<"面积是:"<<Area( )<<endl;
    }
    virtual float Area( ) const { return 0; }
};
```

```
int main( ) {
    Rectangle rect(1,2);
    Shape& rRect = rect;
    Shape * pCircle = new Circle(3);
    rRect.Show( ); //正确输出2
    pCircle->Show( ); //正确输出28.26
    delete pCircle;
}
```

```
class Rectangle: public Shape {
public:
    Rectangle(float w,float h) {
        mWidth = w; mHeight = h;
    }
    virtual float Area( ) const {
        return mWidth*mHeight;
    }
private:
    float mWidth, mHeight;
};
```

```
class Circle: public Shape {
public:
    Circle(float r): mRadius( r ) { }
    virtual float Area( ) const {
        return mRadius * mRadius*3.14;
    }
private:
    float mRadius;
};
```



```
int main( ) {
    Rectangle rect(1,2);
    Shape& rRect = rect;
    Shape * pCircle = new Circle(3);
    rRect.Show( );
    //正确输出2
    pCircle->Show( );
    //正确输出28.26
    delete pCircle;
}
```

```
class Shape{
public:
    virtual ~Shape( ) { }
    void Show( ) const {
        cout<<"面积是:"<<Area( )<<endl;
    }
    virtual float Area( ) const
    { return 0; }
};
```



- ◆ **静态类型**: 变量或表达式在编译时确定的类型
- ◆ 由声明时的类型决定, 与运行时实际对象无关
- ◆ 编译器根据静态类型进行类型检查(判定可访问的成员函数和数据成员)、函数重载解析

如: Child child;

- 指针型: **Parent * pObj = &child;**
- 引用型: **Parent& obj = child;**
- 对象型: **Parent obj = child;**

//对象型中obj的静态、动态一致

- ◆ **动态类型**: 变量或表达式在运行时实际指向的对象的类型

- ◆ 通常适用于指针或引用
- ◆ 动态类型是静态类型的派生类或同一个类
- ◆ 对象型会导致切片, 丢失动态类型信息
- ◆ 父类存在虚函数时, 动态类型才可见

```
int main( ) {  
    Rectangle rect(1,2);  
    Shape& sh1 = rect;  
    Shape * psh2 = new Circle(3);  
  
    rect.Show( ); //非虚函数静态绑定  
    sh1.Show( ); //根据静态类型确定版本  
                //正确输出2?  
    psh2 ->Show( ); //非虚函数静态绑定  
                //正确输出28.26?  
    delete psh2;  
    void userFunc( const Shape & );  
    userFunc( rect );  
}  
  
void userFunc( const Shape & sh ) {  
    sh.Show( ); //非虚函数静态绑定  
    .....  
}
```



根据静态类型确定行为接口是否合法;
动态编联时, 根据动态类型确定行为实现是哪一个。

```
class Shape{
public:
    virtual ~Shape( ) { }
    void Show( ) { //非虚函数
        cout<<"面积是:"<<Area( )<<endl;
    }
    virtual float Area( ) const { return 0; }
};
```

```
void Show ( Shape * const this ) {
    cout <<"面积是:"<< this->Area( )<<endl;
}
```

```
int main( ) {
    Rectangle rect(1,2);
    Shape& sh1 = rect;
    Shape * psh2 = new Circle(3);

    rect.Show( ); // Shape * const this = &rect;

    sh1.Show( ); // Shape * const this = &sh1;
                //正确输出2
    psh2 ->Show( ); // Shape * const this = psh2;
                //正确输出28.26
    delete psh2;
    void userFunc( const Shape & );
    userFunc( rect );
}

void userFunc( const Shape & sh ) {
    sh.Show( );
    .....
}
```



编译过程 (以 `p->Func()` 为例)

- 首先确定 `p` 的静态类型，假如为 `A *`
- 在 `A` 类中，寻找名字为 `Func`，且参数可以匹配的函数
- 若找不到，编译错误。
- 若找到，判断该函数是否为 `virtual` 函数？
- 若不是，编译成 `p->A::Func()`; // 根据 `p` 的静态类型直接绑定 `Func()` 版本
- 若是虚函数，采用动态编联，从而编译成如下形式的函数调用：
 - `(* p->vptr)[index>((void *)p,...);` // 根据 `p` 的动态类型运行时确定
 - `index` 为 `Func` 在 `vtable` 中的序号，在编译时确定。
 - 即在运行时，根据 `vptr` 中的函数入口地址，选择执行函数。

若希望 `p->Func()`; 或 `rObj.Func()`; 合法，需满足：

- ◆ `p / rObj` 的静态类型中必须有匹配的函数 `Func()`;
- ◆ 即使匹配的 `Func()`，永远不被调用，也要有

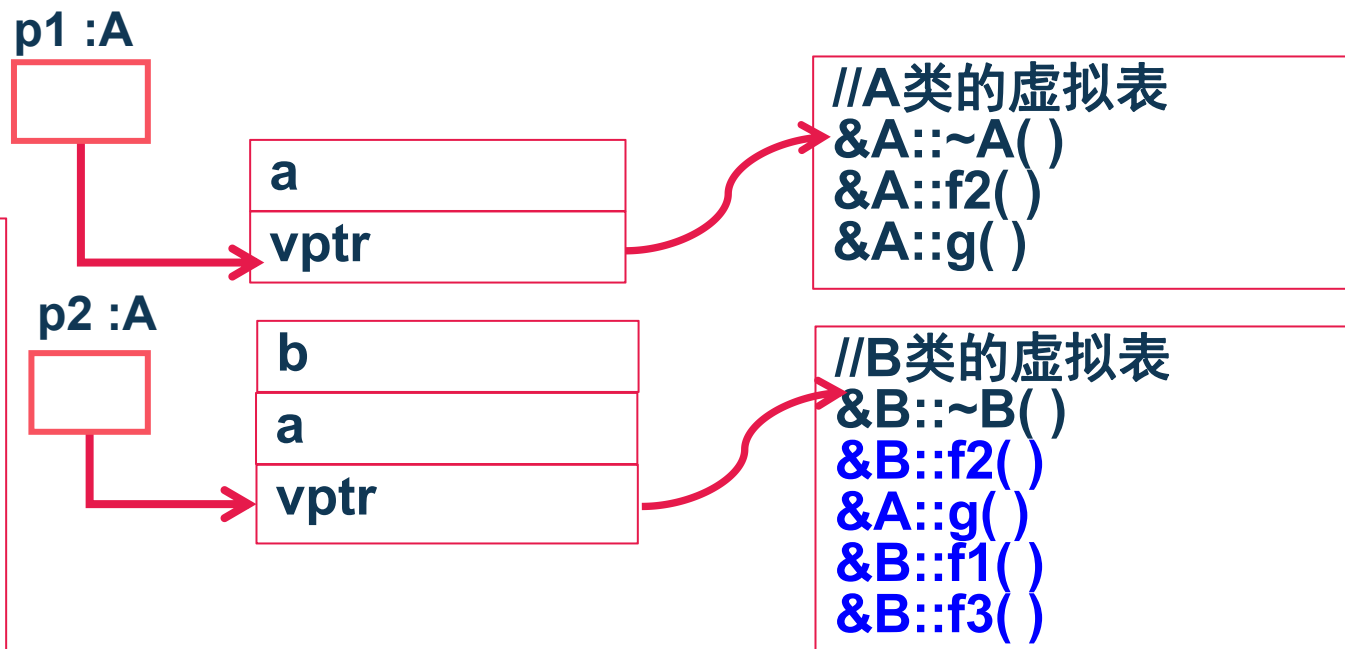

```
class A {
public:
    virtual ~A( ) { }
    virtual void f2( );
    virtual void g( );
    int a;
};

class B:public A {
public:
    virtual void f1( );
    virtual void f2( );
    virtual void f3( );
    int b;
};
```

```
A * p1 = new A;
p1->f2( );
p1->g( );
p1->f1( );
```

```
A * p2 = new B;
p2->f2( );
p2->g( );
p2->f1( );
```

```
B * p3 = new B;
p3->f1( );
delete p3;
```



p1->f2() 编译成 (*p1->vptr)[1]((void *)p1)

p1->g() 编译成 (*p1->vptr)[2]((void *)p1)

p2->f2() 编译成 (*p2->vptr)[1]((void *)p2)

p2->g() 编译成 (*p2->vptr)[2]((void *)p2)

delete p2编译成 ...;(*p2->vptr)[0]((void *)p2)

delete p1编译成 ...;(*p1->vptr)[0]((void *)p1)

```
class A {  
public:  
    virtual ~A( ) { }  
    virtual void f2( );  
    virtual void g( );  
};
```

```
class B:public A {  
public:  
    virtual ~B( ) { }  
    virtual void f1( );  
    virtual void f2( );  
    virtual void f3( );  
};
```

```
void Func(A* p){  
    p->f2( );  
    p->a( );  
    p->f1( );  
}
```

```
int main(){  
    A * p1 = new A;  
    Func(p1);  
  
    A * p2 = new B;  
    Func(p2)  
  
    delete p2;  
    delete p1;  
  
    return 0;  
}
```

p->f2() 编译成 (*p->vptr)[1]((void *)p)
p->g() 编译成 (*p->vptr)[2]((void *)p)

delete p2编译成 ...;(*p2->vptr)[0]((void *)p2)
delete p1编译成 ...;(*p1->vptr)[0]((void *)p1)

A * p1 = new A;
Func(p1);

vptr

A * p2 = new B;
Func(p2)

vptr

//A类的虚拟表
&A::~~A()
&A::f2()
&A::g()

//B类的虚拟表
&B::~~B()
&B::f2()
&A::g()
&B::f1()
&B::f3()

```
class Date {
public:
    Date(int y,int m,int d );
    virtual ~Date( ) { }
    bool    IsLeap( ) const;
    virtual void Display( ) const { }
protected:
    int mYear;
    int mMonth;
    int mDay;
};

void ShowDate(Date * p) { p->Display( ); }
```

```
void ShowDate(Date * p) ;

Date * pDate1 = new USADate(2008,12,31);
ShowDate(pDate1); // 12/31/2008
delete pDate1;

Date * pDate2 = new CHINADate(2008,12,31);
ShowDate(pDate2) // 2008年12月31日
delete pDate2;
```

```
class CHINADate: public Date {
public:
    CHINADate(int y,int m,int d)
        :Date(y,m,d) { }
    virtual void Display( ) const {
        cout<<mYear<<"年" ;
        cout<<mMonth<<"月" ;
        cout<<mDay<<"日" <<endl;
    }
protected: // other
};
```

```
class USADate: public Date {
public:
    UsaDate(int y,int m,int d)
        :Date(y,m,d) { }
    virtual void Display( ) const {
        cout<<mMonth<<"/";
        cout<<mDay<<"/";
        cout<<mYear<<endl;
    }
    // ....
};
```

- ◆ 虚函数中访问的非虚函数
静态编联，使用本地版本
- ◆ 非虚函数中访问的虚函数
动态编联
- ◆ 虚函数中访问的虚函数
动态编联
- ◆ 构造函数和虚函数
 - 构造函数不能为虚函数
 - 调用的虚函数采用静态编联，使用本地版本
- ◆ 析构函数和虚函数
 - 析构函数可以为虚函数
 - 若类中含有虚函数，那么析构函数也应为虚函数。
 - 调用的虚函数采用静态编联，使用本地版本

```
class A {
public:
    virtual ~A() {}
    virtual void f() { g(); }
    virtual void g() { k(); t(); }
    void k() { t(); }
    virtual void t();
};

class B:public A {
public:
    virtual void f() { g(); k(); }
    void k() { h(); }
    void h();
    virtual void t();
}

A * p1 = new A;
p1->XXX(); //XXX是函数名替代
A * p2 = new B;
p2->XXX(); //XXX是函数名替代
```

```
class A {  
public:  
    A():mA(100) { f( ); }  
    virtual ~A() { f( ); }  
    virtual void f( )  
        { cout<< mA; }  
private:    int mA;  
};
```

```
class B:public A {  
public:  
    B():mB(200) { }  
    virtual ~B() { f( ); }  
    virtual void f( )  
        { cout<<mB; }  
private:  
    int mB;  
};
```

◆ B b;

先构造父类A，调用本地的f()，即A::f()

◆ B b;

释放时，先析构B类，执行B::~~B()；然后要析构A类，需要执行A::~~A()，只能调用本地的f()，即A::f()；



虚函数的访问(例)

```
class Base {
public:
    Base( ) { vf( ); }
    virtual ~Base( ) { vf( ); }
    virtual void vf( ) {
        cout<<"Base::vf( )" << endl;
    }
    virtual void vg( ) {
        cout<<"Base::vg( )" << endl;
        vf( );
        nvh( );
    }
    void nvh( ) {
        cout<<"Base::nvh( )" << endl;
        vf( );
    }
};
```

```
class Derived: public Base {
public:
    Derived( ) { vf( ); }
    virtual ~Derived( ) { vf( ); }
    virtual void vf( ) {
        cout<<"Derived::vf( )" << endl;
    }

    void nvh( ) {
        cout<<"Derived::nvh" << endl;
        vf( );
    }
    virtual void My( ) { }
};
```

```
Base * p = new Derived;
p->vf( );
p->vg( );
p->nvh( );
delete p;
```



私有的虚函数访问(区别于其它语言)

```
class A {
public:   virtual ~A( ) { }
        virtual void vPub1( ) {
            cout << "A::vPub1( )" << endl;
            vPriv( );
            vProt( );
        }
        virtual void vPub2( )
        { cout << "A::vPub2( )" << endl; }
protected:
        virtual void vProt( )
        { cout << "A::vProt()" << endl; }
private:
        virtual void vPriv()
        { cout << "A::vPriv()" << endl; }
};
```

```
class B: public A {
public:
        virtual ~B( ) {}
        virtual void vPub2( ) {
            cout << "B::vPub2( )" << endl;
            vPriv( );
            vProt( );
        }
private:
        virtual void vPriv() {
            cout << "B::vPriv()" << endl;
        }
};
```

```
int main( ) {
    A * pA= new B;
    pA->vPub1( );
    pA->vPub2( );
    delete pA;
    return 0;
}
```

```
A::vPub1( )
B::vPriv()
A::vProt()
-----
B::vPub2( )
B::vPriv()
A::vProt()
```



◆ 具体类和抽象类

- 具体类：可以实例化
- 抽象类：为子类提供更高层次的抽象，本身不能被实例化，但后裔类可以实例化。

◆ 抽象类的定义

- 含有一个或多个纯虚函数。
- 纯虚函数格式 (一定是成员函数) :
`virtual Return Type Func(... ) [const] = 0;`
- 纯虚函数的访问控制可任意
- 具体类的子类可以是具体类或抽象类
- 抽象类的子类可以是具体类或抽象类
- 纯抽象类：除静态、构造、析构等函数均为纯虚函数。
- 纯虚定义:对纯虚函数给出缺省实现(定义)，特别地，纯虚析构函数必须有实现

◆ C++中的接口类

- 可以用纯抽象类表示，通常均为public成员，且没有任何非静态的数据成员。



```
class Shape { //抽象类
public:
    virtual ~Shape() { }
    void Show( ) const
    { cout<<"面积:"<<Area( )<<endl; }
    virtual float Area( ) const = 0;
};
```

```
class Rectangle:public Shape { //具体类
public:
    virtual ~Rectangle( ) { }
    virtual float Area( ) const
    { return w*h; }
private:
    int w; int h;
};
```

```
//shape.cpp中给出纯虚定义
float Shape::Area( ) const {
    return 0;
}
```

注意：纯虚定义只能在类外给出

```
class Line:public Shape { //具体类
public:
    virtual ~Line( ) { }
    //若Shape::Area有纯虚定义，
    //子类可按需复用
    virtual float Area( ) const
    { return Shape::Area(); }
};
Line ml;
ml.Shape::Area();
```



```
class Shape1 { //抽象类
public:
    virtual ~Shape1() { }
    void Show( ) const
        { cout<<"面积:"<<Area( )<<endl; }
    virtual float Area( ) const = 0;
};
```

```
class Polygon:public Shape1 { };
    //可以仍是抽象类
class Rectangle:public Shape1 { //具体类
public: virtual float Area( ) const { }
};
class Farm:public IAreable { //具体类
public: virtual float Area( ) const { }
    ...
};
```

```
class IAreable { //接口类
public:
    virtual ~IAreable() { }
    virtual float Area() const = 0;
};
```

//派生接口类

```
class ICountry: public IAreable {
public:
    virtual int Population() const = 0;
};
```

```
class ICloneable { //接口类
public:
    virtual ~ICloneable( ) { }
};
```



typeid(exp) 或 typeid(type_name),
返回 const type_info 对象

- 在没有虚函数时，仅仅返回静态类型对应的type_info 对象
- 在有虚函数时：
- 若exp是指针，则返回静态类型对应的type_info 对象；
- 若exp是类类型(对象/引用)，且含有虚函数，则返回动态类型对应的type_info 对象。例：

```
#include <typeinfo>
class A { /*含虚函数*/ };
class B:public A { };
class C:public A { };
A * p1 = new B;
typeid(p1) == typeid(A*) // TRUE
typeid(p1) == typeid(B *) // FALSE
typeid(*p1) == typeid(A) // FALSE
typeid(*p1) == typeid(B) // TRUE
const type_info & type = typeid(p1);
const char* tpName = type.name();
```



dynamic_cast<T>(exp)

- T只能是指针或引用型
- 若T为指针，成功则返回T类型指针；失败返回nullptr
- 若T为引用，成功则返回T类型引用；失败产生bad_cast异常

```
class A { };
```

```
class B:public A { };
```

```
class C:pubilc A { };
```

```
A * p1 = new B;  A * p2 = new C;
```

```
B * pb = dynamic_cast<B*>(p1); //成功pb非空
```

```
if (pb) {  pb->Func( ); }
```

```
B & b = dynamic_cast<B&>(*p1); //成功则继续，失败则产生异常
```

```
b.Func( );
```



本章结束